DA5400: Foundations of Machine Learning

Topic 5: Classification

WSAI IITM

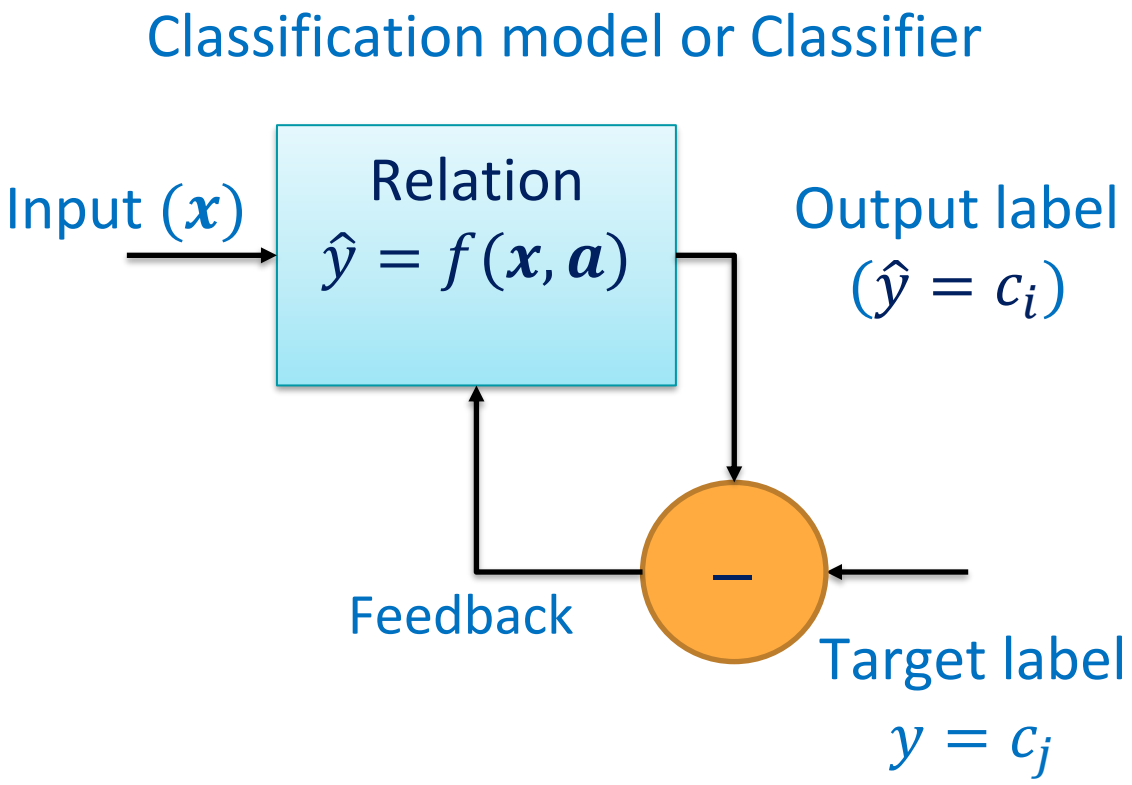
Contents

- Introduction to Classification
- Types of Classification Models
- Classification Algorithms
  - K-Nearest Neighbour classifier
  - Logistic Regression
  - Naïve Bayes Classifier
- Metrics to Evaluate Classification Models

Introduction to Classification

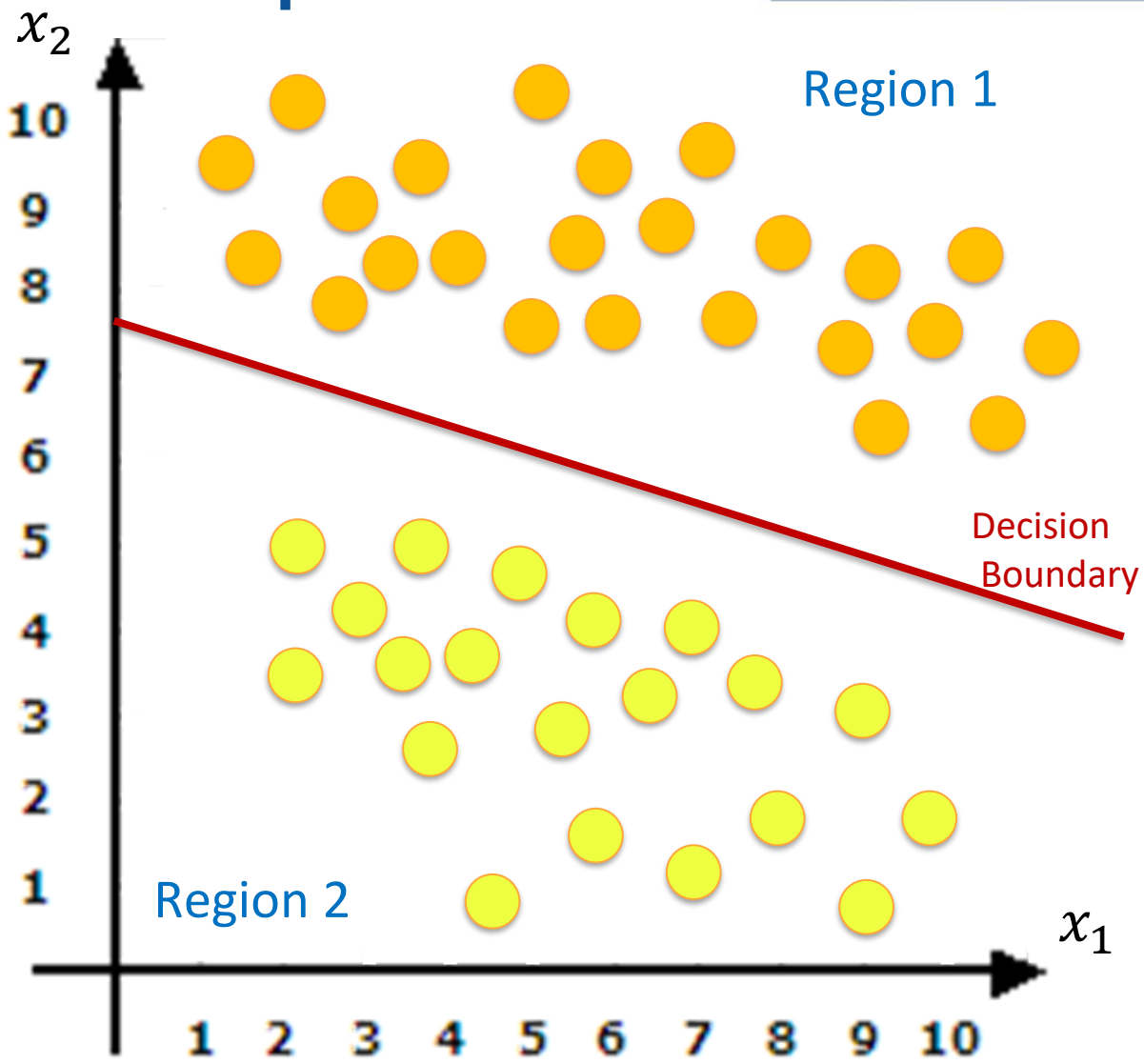
Classification

- A supervised learning task in which the output variables ($\boldsymbol{y}$) to be predicted are discrete values referred to as class labels
- Examples:
  - Classifying emails as spam or not based on subject or body
  - Given handwritten character images, classify it to be one of the known characters
  - Given details of a financial transaction, classifying it to be genuine or fraudulent
  - Given the details of a tumour, classifying it to be benign or malignant
  - Given the parameters of a process in an industry, classifying whether there is fault in the process or not



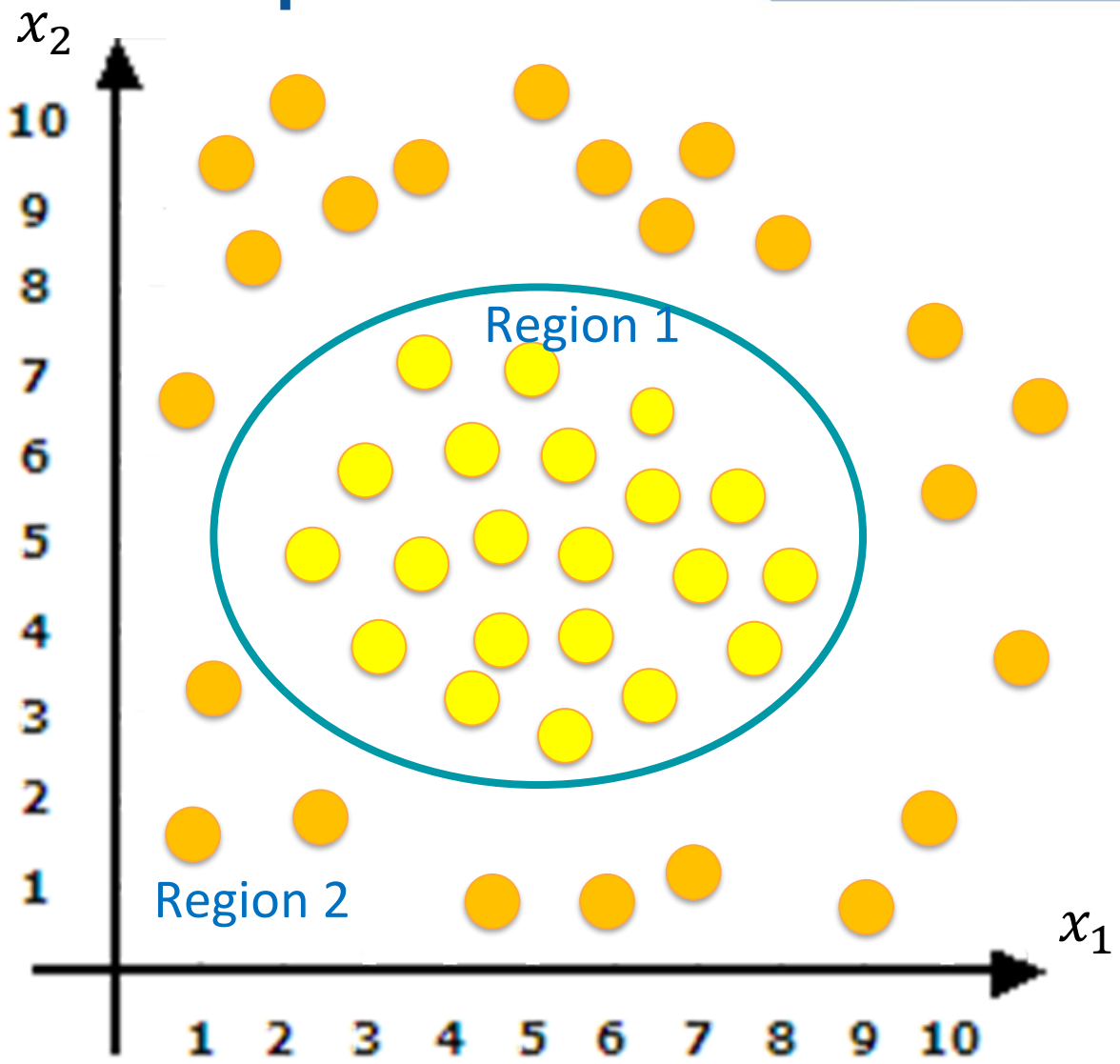
Classification: Feature Space Perspective

- A classifier divides the feature space into a set of regions each having a class label associated with it
- Boundaries between these regions can be straight, curved, smooth or rough depending on the prediction function
- Referred to as decision boundaries



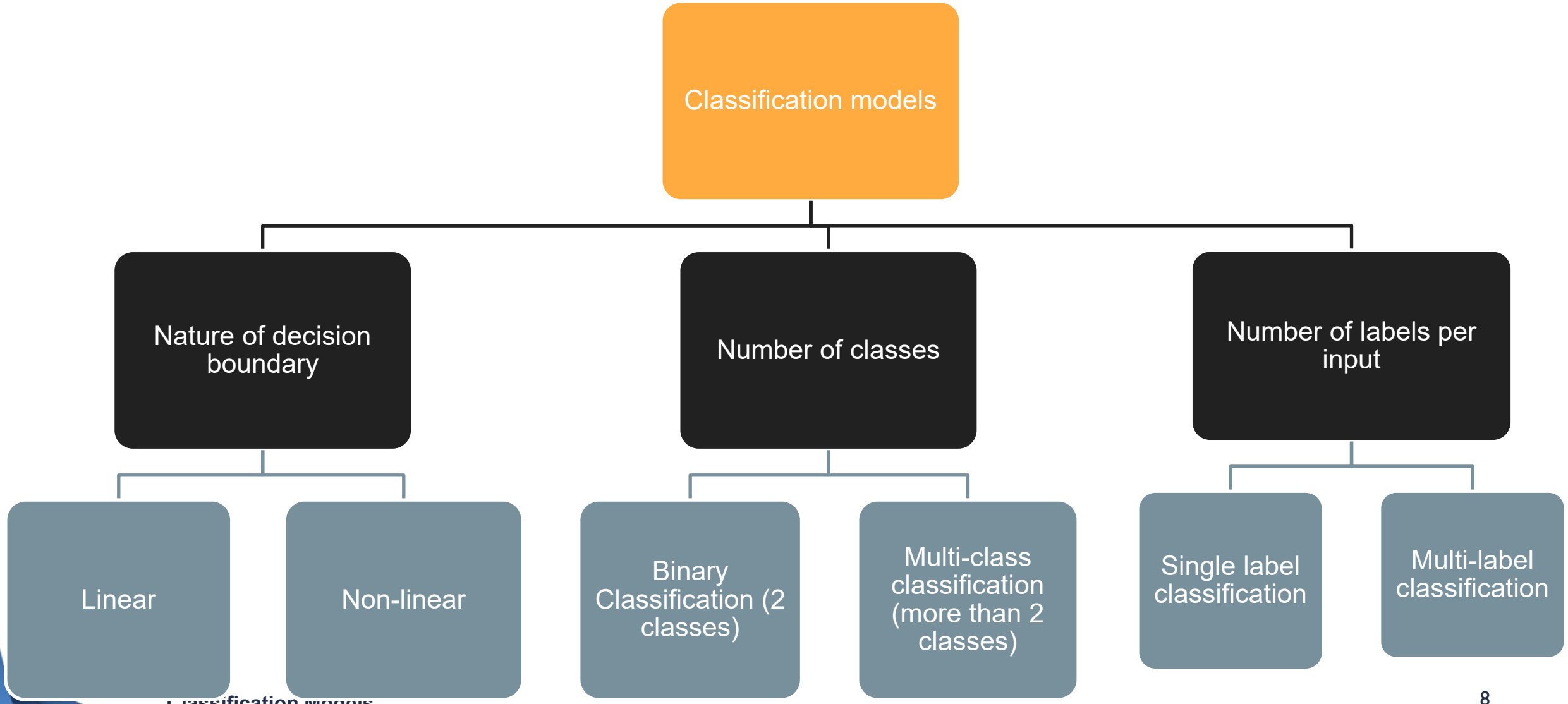
Classification: Feature Space Perspective

- A classifier divides the feature space into a set of regions each having a class label associated with it
- Boundaries between these regions can be straight, curved, smooth or rough depending on the prediction function
- Referred to as decision boundaries
- Class predictions will be based on the region in which the new input lies

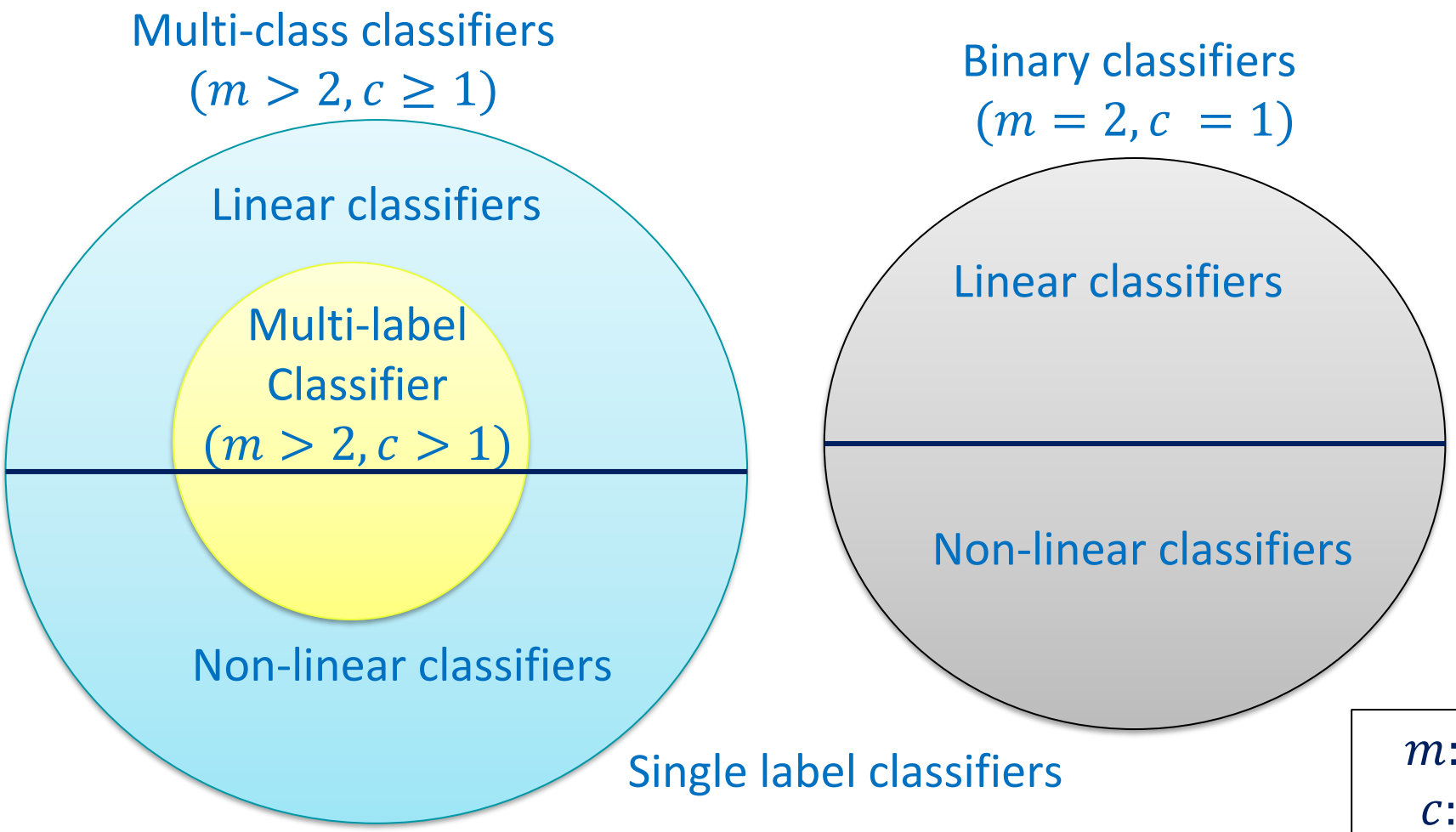


Types of Classification Models

Types of Classification Models



Types of Classification Models



m: Total no of classes
c: No of class labels predicted per data point

Soft vs Hard Predictions in Classification

- Soft Prediction: Prediction that gives the probability of a data point belonging to a each of the target classes
- Hard Prediction: Prediction that gives the exact class that the data point might belong to
- Soft prediction gives more information on the confidence of the classifier in the prediction made
- Soft prediction can be turned into a hard prediction by setting a probability threshold

Eg: Predicting whether a transaction is fraud or genuine

p_0: Probability that transaction is genuine

p_1: Probability that transaction is fraud

Soft predictions:

Sample 1: $\begin{bmatrix} p_0 \\ p_1 \end{bmatrix} = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}$

Sample 2: $\begin{bmatrix} p_0 \\ p_1 \end{bmatrix} = \begin{bmatrix} 0.55 \\ 0.45 \end{bmatrix}$

Hard predictions with Threshold = 0.5:

Sample 1 – Genuine

Sample 2 - Genuine

Hard predictions with Threshold = 0.6:

Sample 1 – Genuine

Sample 2 - Fraud

Classification Algorithms

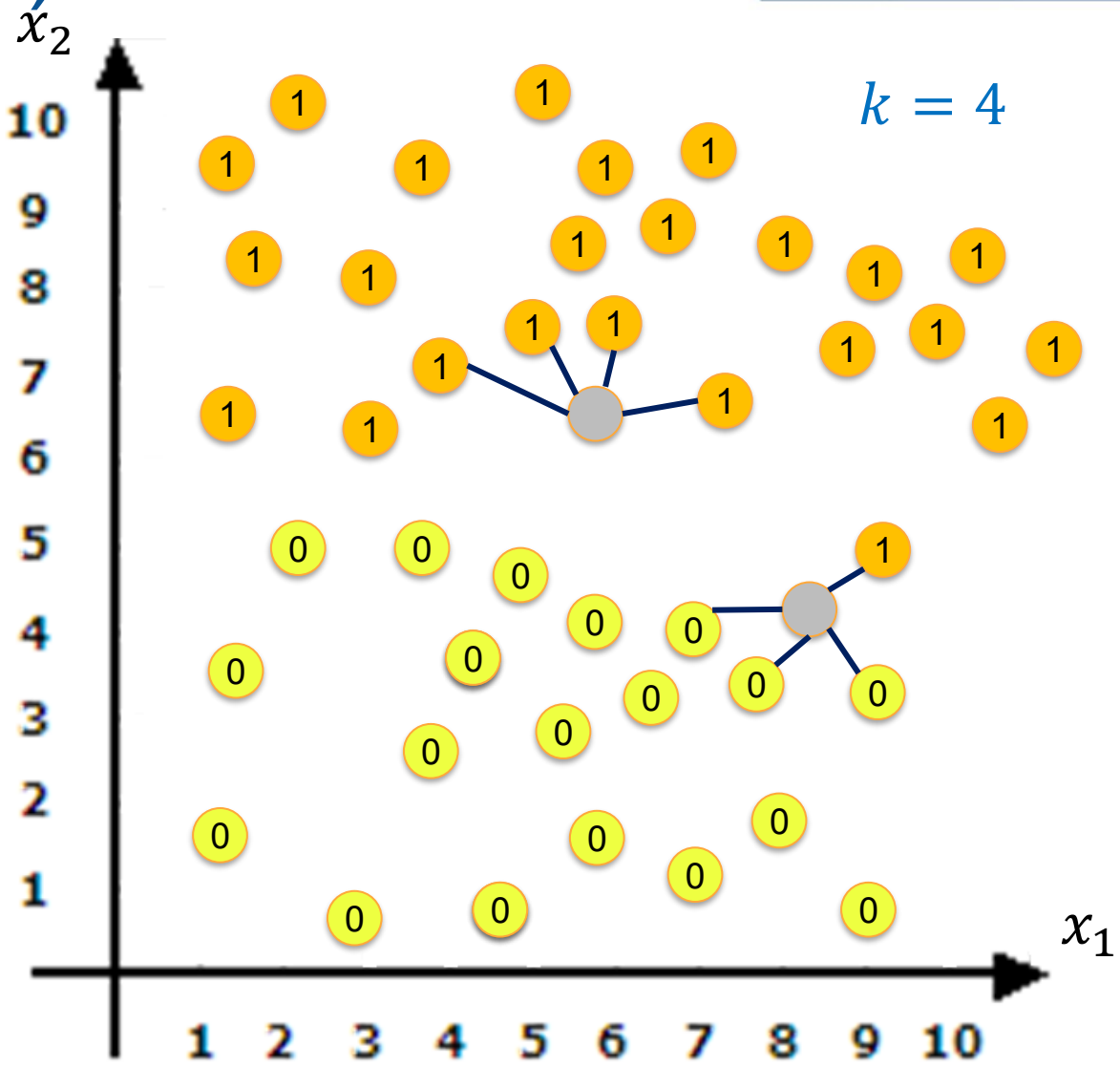
Some Points to Note

- Pre-processing techniques for regression can also be used for classification
- Binary Classification:
  - Classes are referred to as positive (1) and negative (0)
- Model Validation:
  - Model validation techniques used in regression can be used for classification
- Accuracy in prediction:
  - Number of samples correctly predicted by total number of samples used in prediction $\left(\frac{N_c}{N}\right)$
- Error in prediction:
  - Number of samples which are mis-classified i.e., class predicted is different from actual class
  - % error can be taken by dividing the number of mis-classified samples by total number of sampled used in prediction $\left(1 - \frac{N_c}{N}\right)$

K-Nearest Neighbour Classifier

k-Nearest Neighbour (kNN) Classifier

- Data points are classified based on their neighbours in the feature space
- Majority of the classes of the k-nearest neighbours is taken as the class for a data point
- k is a hyper-parameter to be decided by the user
- No model is learnt and stored in this classifier – lazy classifier
- It is non-parametric and makes no assumptions on data distribution



k-Nearest Neighbour (kNN) Classifier

- Training data: $\{\boldsymbol{x}_i, y_i\}_{i=1}^{N}$
- Distance metric: $d(\boldsymbol{x}, \boldsymbol{x}_i)$
- $\mathcal{N}_k(\boldsymbol{x})$: Indices of k nearest neighbours of $\boldsymbol{x}$

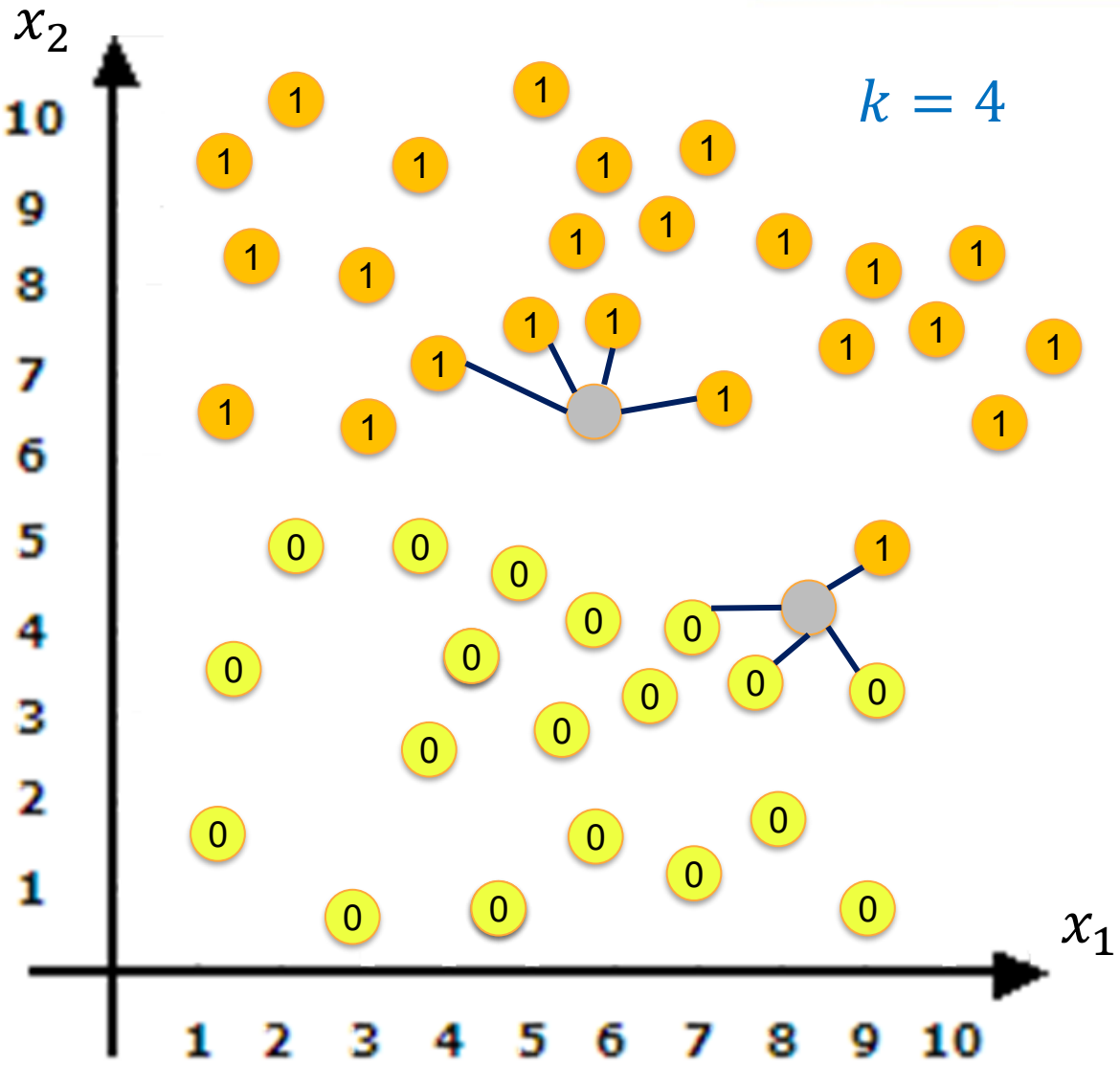
$$\mathcal{N}_k(\boldsymbol{x}) = \underset{S \subset \{1,2,\ldots,N\}}{\operatorname{argmin}} \sum_{i \in S} d(\boldsymbol{x}, \boldsymbol{x}_i)$$

- Hard prediction:

$$\hat{y}_i(\boldsymbol{x}) = \underset{c}{\operatorname{argmax}} \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} 1[y_i = c]$$

- Soft prediction:

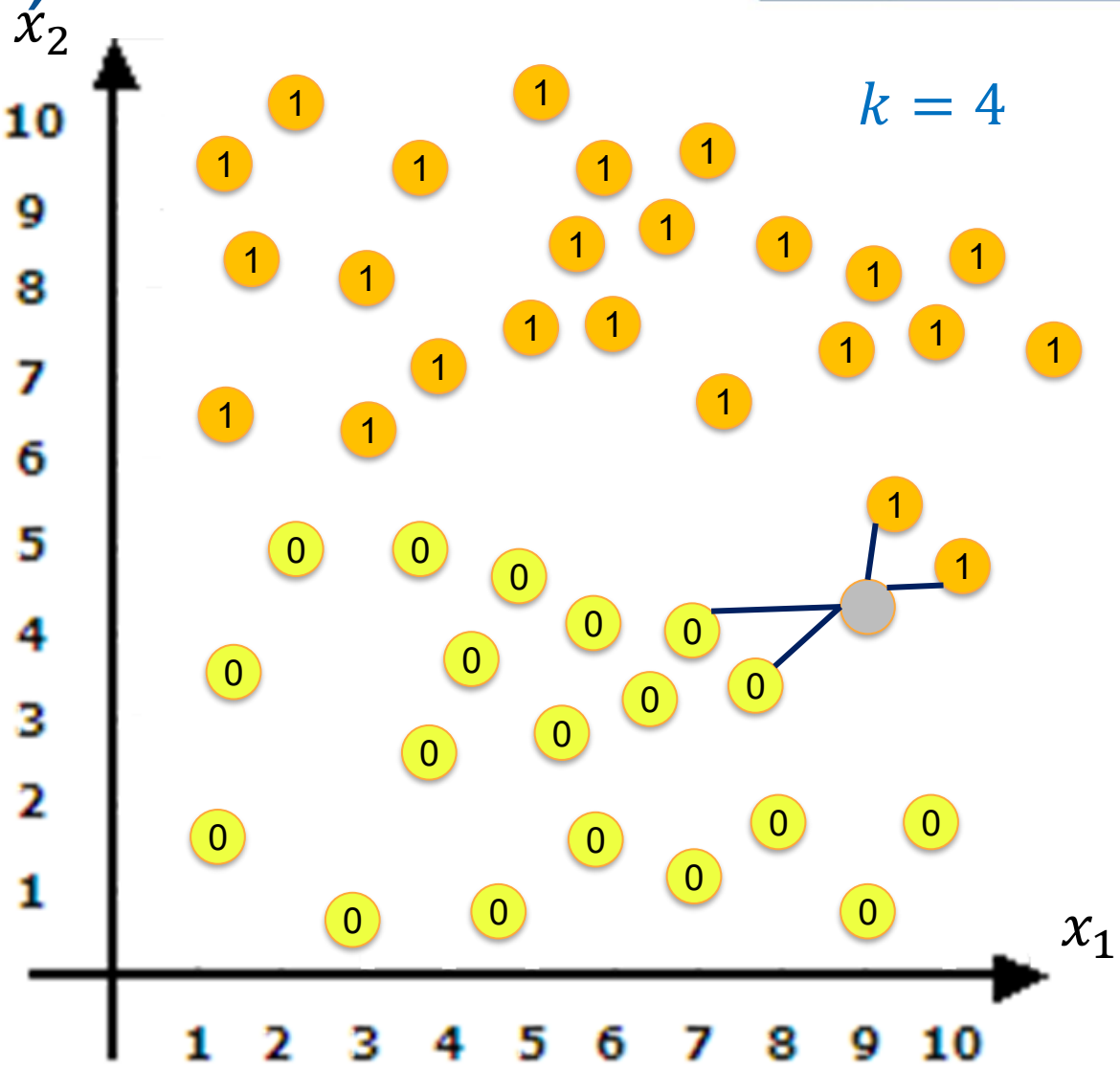
$$\hat{p}(c|x) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\boldsymbol{x})} 1[y_i = c]$$



k-Nearest Neighbour (kNN) Classifier

- Weighted kNN: While taking majority among k neighbours, they can be weighed by the reciprocal of their distance $w_i = \frac{1}{d(\boldsymbol{x}, \boldsymbol{x_i})}$

- k can be chosen through:
  - Trial and error by observing the error in prediction for different values of k

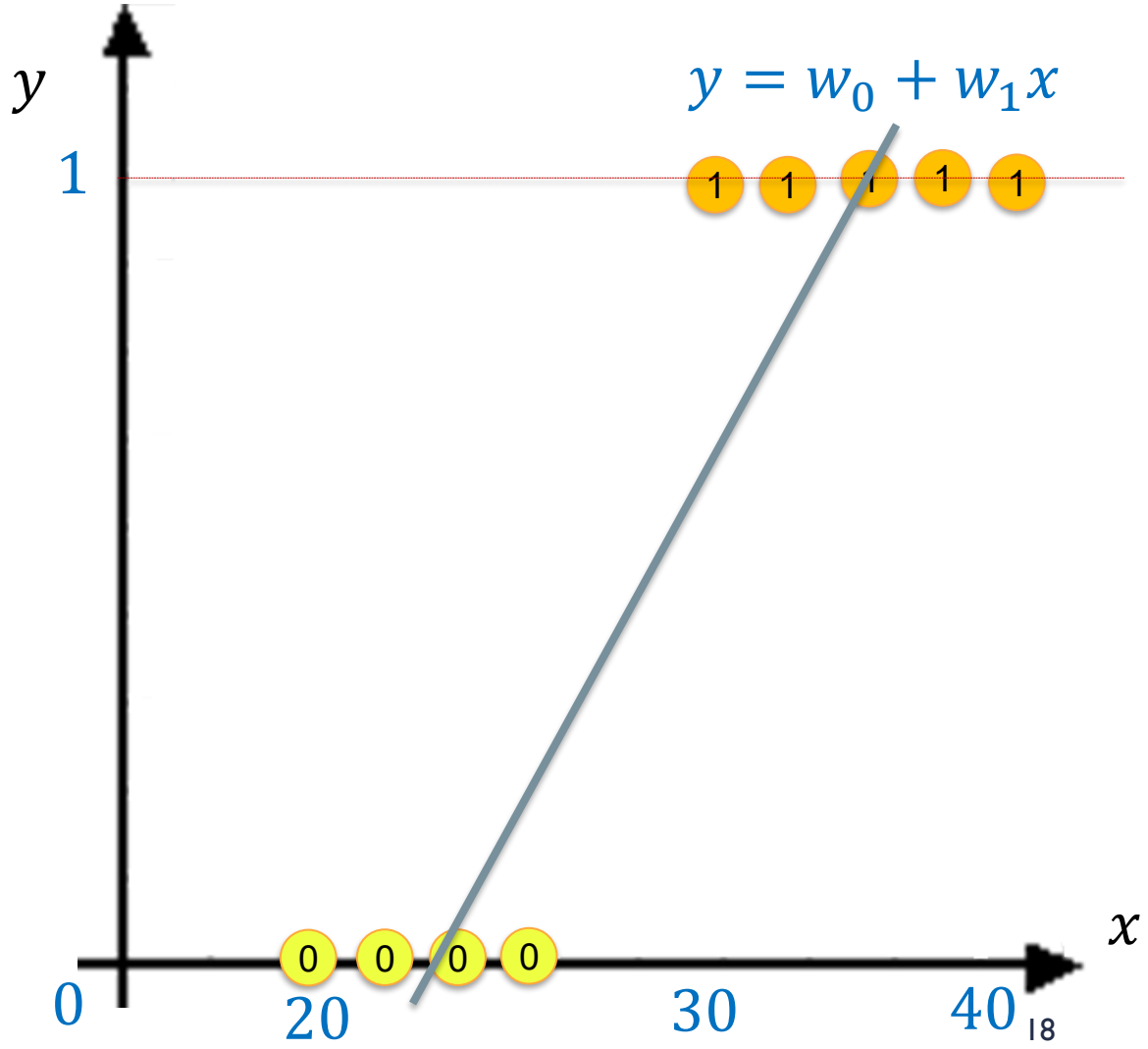


Logistic Regression

Logistic Regression - Motivation

x	20.5	23	24.6	25	31.5	33	35	38.6	40
y	0	0	0	0	1	1	1	1	1

- Objective: To make soft predictions i.e., probability of a sample belonging to a particular class
- Question: Can linear regression be used to predict classes (y) given x ?
- Eg: Binary Classification - To predict whether a person is normal or obese based on BMI
- Suppose linear regression is performed to predict y in this case
- Issue: Output is not guaranteed to lie between 0 and 1 and spans $(-\infty, \infty)$
- Question: Can the output of regression be transformed into a probability value?



Logistic Regression – Logit Link

- Goal: Model $p = P(y = 1|x)$
- Idea: Model the probability via function that always outputs (0,1)
- Logit function transforms (0,1) to $(-\infty, \infty)$

$$\text{logit}(p) = \log \frac{p}{1-p}$$

- Therefore, logit function acts as a natural bridge between linear prediction (z) and probabilistic prediction (p)

$$\log \frac{p}{1-p} = z = w_0 + w_1 x$$

$$p = \frac{1}{1 + e^{-z}}$$

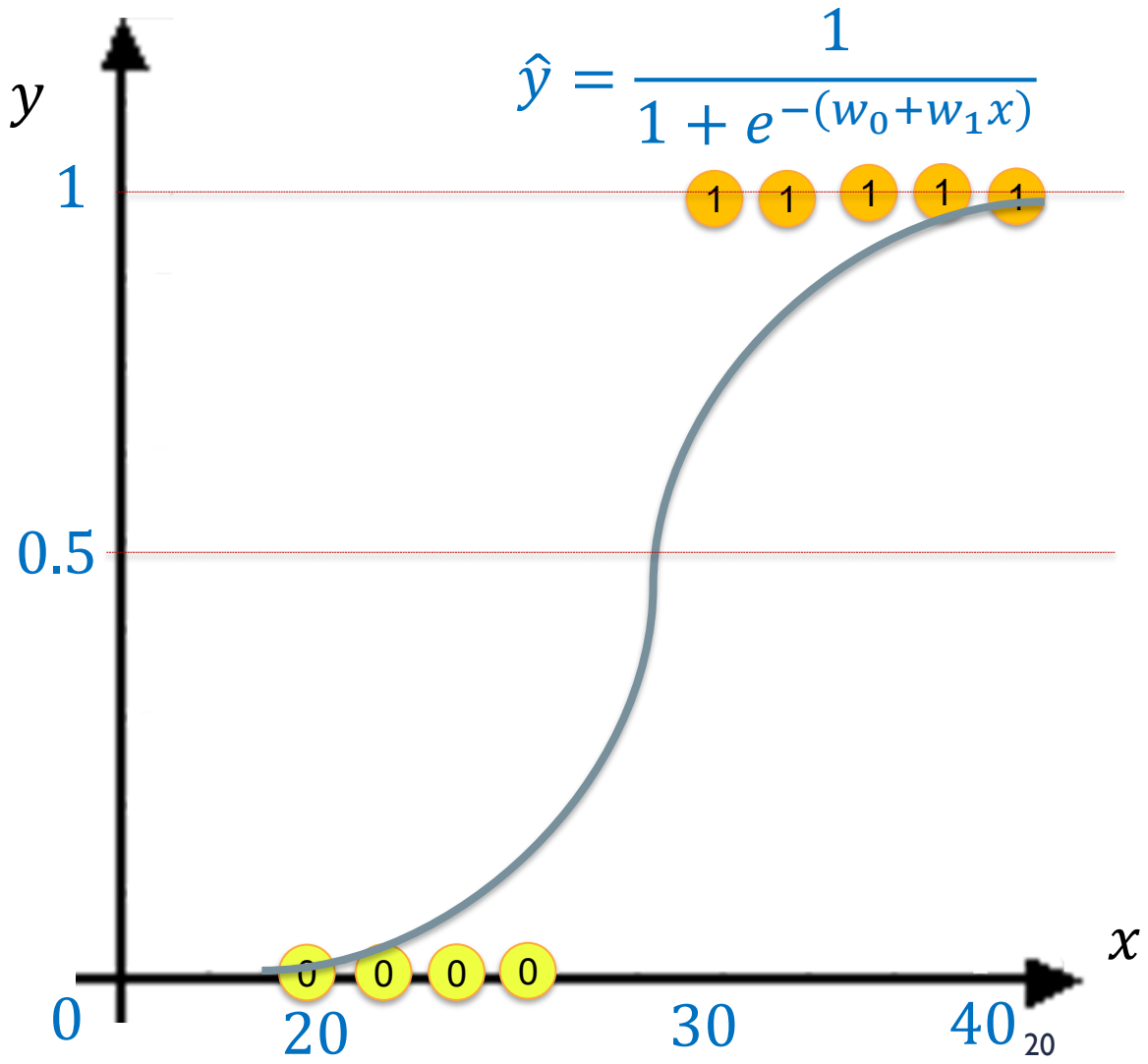
Logistic Regression – Sigmoid Function

- Sigmoid function maps any value (z) to a value between 0 and 1 (S-shaped)

$$p = \frac{1}{1 + e^{-z}}$$

$$\Pr(y = 1|x) = \frac{1}{1 + e^{-(w_0 + w_1 x)}}$$

- Objective: Parameters $w_0, w_1, \ldots$ are to be obtained such that predicted y is close to actual y



Logistic Regression – Learning Parameters

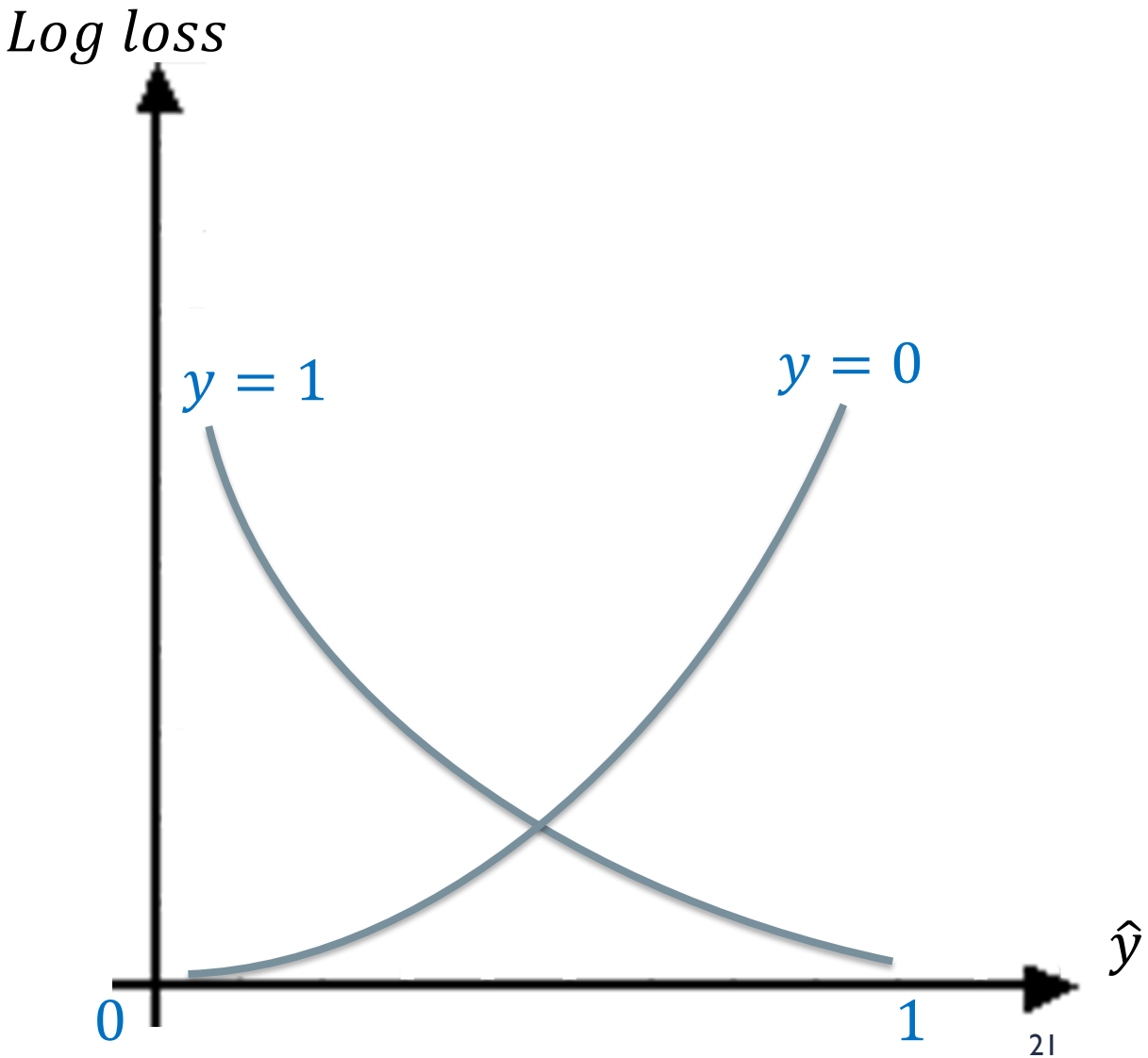
- Known: Data, Model structure

$$\hat{y} = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + \cdots w_D x_D)}}$$

- Squared error loss cannot be used because it will have many local minima
- Loss function: Log loss is used because it goes to zero when prediction matches actual output

$$log\ loss = -\ln \hat{y} \quad if\ y = 1$$
$$= -\ln(1 - \hat{y}) \quad if\ y = 0$$
$$l(y, \hat{y}) = -(y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})$$

- To obtain the parameters $w_0, w_1, \ldots, w_D$ log loss is minimised over all samples using gradient descent technique



Logistic Regression – Gradient Descent

- Objective:

$$J(w) = \frac{1}{N}\sum_{i=1}^{N} -(y_i \ln \hat{y}_i + (1 - y_i)\ln(1 - \hat{y}_i))$$

- Gradients:

$$\frac{\partial J}{\partial w} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)x_i$$

$$\nabla_w J = \frac{1}{N} X^T (y - \hat{y})$$

- Gradient Descent update rule:

$$w_{new} = w_{old} - \alpha \frac{\partial J}{\partial w}$$

Logistic Regression – Decision Boundary

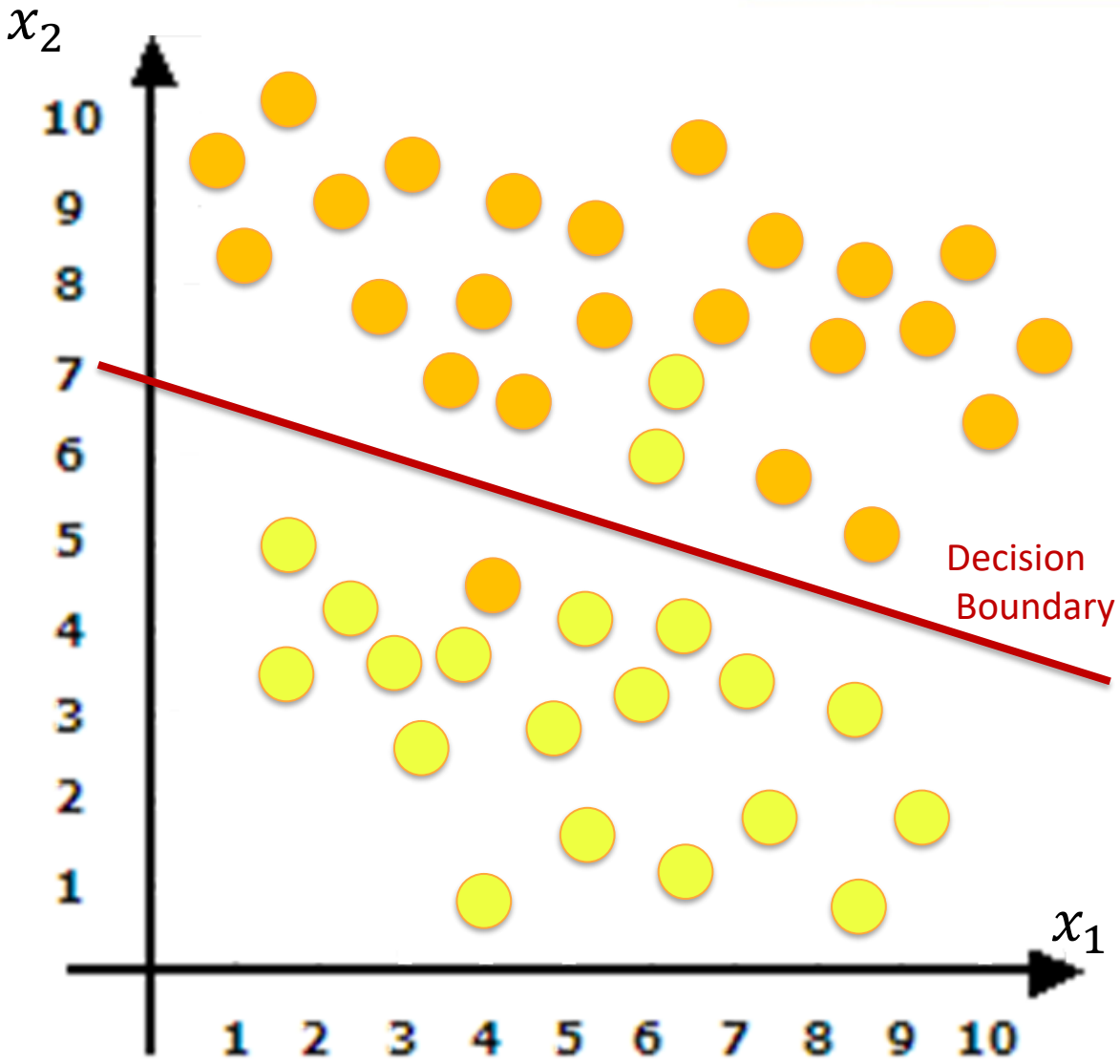
- To find out the decision boundary given by logistic regression, set the probability to threshold value (0.5)

$$\hat{y} = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + \cdots w_D x_D)}} = 0.5$$

$$e^{-(w_0 + w_1 x_1 + \cdots w_D x_D)} = 1$$

$$w_0 + w_1 x_1 + \cdots w_D x_D = 0$$

- Interpretation: Decision boundary is a line or a hyperplane which minimises the loss



Multiclass Logistic Regression

Multiclass Logistic Regression - Motivation

- Objective: To predict the probability that a data point belongs to each of the m classes
- All the m probabilities should add up to 1
- Idea: To use a separate set of parameters for each class
- Learn parameters $w_{0i}, \dots w_{Di}$ for $i = 1, \dots m$
- Rest of the steps from calculating loss to performing gradient descent remain the same
- Question: How to ensure that the sum of probabilities of all m classes adds up to 1 ?
  - Replace Sigmoid with Softmax function

p classes

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{bmatrix} \rightarrow \hat{y} = \begin{bmatrix} \widehat{y_1} \\ \widehat{y_2} \\ \vdots \\ \widehat{y_m} \end{bmatrix}$$

$$\widehat{y_1} + \widehat{y_2} + \cdots + \widehat{y_m} = 1$$

$$z_{n,c} = w_{0i} + w_{1i}x_1 + \cdots w_{Di}x_D$$

where $i = 1,2, \dots, m$

Softmax Function

- Sigmoid function gives a value between 0 and 1, and can be used for binary classification
- However, sigmoid cannot output multiple probability values which add upto 1 (multi-class)
- Softmax function is an extension of sigmoid function
- Softmax calculates the relative probabilities of multiples classes and ensures that total probability is 1

Input to softmax function

$$\boldsymbol{z} = [z_1 \quad z_2 \quad \dots \quad z_m]$$

Total Probability = 1

Prob. of class 0	Prob. of class 1

Binary Classification

Total Probability = 1

Prob. of class 1	Prob. of class 2	Prob. of class 3

Multi-class Classification

$$\widehat{y_{i,c}} = \frac{e^{z_{i,c}}}{\sum_{j=1}^{m} e^{z_{i,j}}}$$

Cross Entropy Loss

- For classification problems with more than two classes, cross entropy loss is used
- Cross entropy loss: $\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{m} y_{i,c} \ln \widehat{y_{i,c}}$
- Consider a classification problem with output having 4 classes
- For each input, the probability of the input belonging to each of the four classes is predicted (softmax probability)
- Consider a sample belonging to class 1

$$\boldsymbol{y} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \qquad \hat{\boldsymbol{y}} = \begin{bmatrix} 0.7 \\ 0.25 \\ 0.03 \\ 0.02 \end{bmatrix}$$

Desired output Predicted output

$$\mathcal{L} = -(1 * \ln(0.7) + 0 * \ln(0.25)$$

$$+0 * \ln(0.03) + 0 * \ln(0.02))$$

$$\mathcal{L} = -(-0.357)$$

$$\mathcal{L} = 0.357$$

Multiclass Logistic Regression - Summary

Collect Data with each input mapped to one of p classes $(\boldsymbol{X}, \boldsymbol{Y})$

Randomly initialize the parameters $w_{0i}, \dots w_{Di}$ for $i = 1, \dots m$

Compute the linear combinations $z_i = w_{0i} + w_{1i}x_1 + \cdots w_{Di}x_D$ where $i = 1,2, \dots, m$

Compute softmax probabilities $\widehat{y_{i,c}} = \frac{e^{z_{i,c}}}{\sum_{j=1}^{m} e^{z_{i,j}}}$

Compute the loss across all samples $\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{m} y_{i,c} \ln \widehat{y_{i,c}}$

Minimise the total loss by updating the parameters through gradient descent

Performance Metrics for Classification

Errors in Classification

- Ideally, the classifier should predict the class labels of all the samples without any errors
- Since errors are inevitable, one needs to track for how many samples, the class labels are predicted correctly and for how many the predicted labels are erroneous – Confusion Matrix

Confusion Matrix

- A table or a matrix with different combinations of predicted and actual values for different classes for a given probability threshold
- For a binary classification task, the confusion matrix is:

Predicted Values \ Actual Values	Positive (Class 1)	Negative (Class 0)
Positive (Class 1)	True Positives (TP)	False Positives (FP)
Negative (Class 0)	False Negatives (FN)	True Negatives (TN)

Confusion Matrix

Predicted Values	Actual Values: Positive (Class 1)	Actual Values: Negative (Class 0)
Positive (Class 1)	True Positives (TP)	False Positives (FP)
Negative (Class 0)	False Negatives (FN)	True Negatives (TN)

- TP: Sample Class is predicted to be positive (1) and its actual class is also positive (1)
- TN: Sample Class is predicted to be negative (0) and its actual class is also negative (0)
- FP: Sample Class is predicted to be positive (1) but its actual class is negative (0)
- FN: Sample Class is predicted to be negative (0) but its actual class is positive (1)

Building a Confusion Matrix: Example

Sample ID	Predicted Class	Actual Class
1	0	0
2	1	0
3	1	1
4	0	0
5	1	1
6	1	1
7	1	0
8	0	1
9	0	0
10	1	1

Predicted Values	Actual Values: Positive (Class 1)	Actual Values: Negative (Class 0)
Positive (Class 1)	4	2
Negative (Class 0)	1	3

Performance Metrics

- Different performance metrics for classifiers have been defined based on the values in the confusion matrix
  - Accuracy
  - Precision
  - Recall
- Other performance metrics have also been defined including:
  - F1 Score
  - Balanced Accuracy
  - ROC-AUC

Performance Metrics: Accuracy

- Simplest of performance metrics used in classification
- Suitable for both binary and multi-class classification

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

- Accuracy is 1 when FP and FN are zero
- Accuracy is a good choice for evaluating a classifier when the samples are well balanced and not skewed towards one class
- Accuracy is not useful when the dataset of largely imbalanced

Performance Metrics: Precision

- Indicates what proportion of the predicted positives are actually positive

$$Precision = \frac{TP}{TP + FP}$$

- Precision is 1 when there are no false positives
- Precision is a good choice for evaluating a classifier when cost of a false positive is high and cost of a false negative is low
- Example: Predicting whether an email is spam or not

Performance Metrics: Recall

- Indicates what proportion of actual positives are correctly classified i.e., calculates percentage of actual positives a model correctly identified

$$Recall = \frac{TP}{TP + FN}$$

- Recall is 1 when there are no false negatives
- Recall is a good choice for evaluating a classifier when cost of a false positive is low and cost of a false negative is high
- Example: Predicting whether a person has a disease or not

Performance Metrics: F1 Score

- F1 score is the harmonic mean of precision and recall which takes a value between 0 and 1

$$F1\ score = 2 * \frac{precision * recall}{precision + recall}$$

- F1 score maintains a balance between precision and recall i.e., give equal weightage to both
- Useful when we want a classifier model with both good precision and recall

Performance Metrics: Balanced Accuracy

- Balanced accuracy is the mean of sensitivity and specificity

$$Sensitivity = Recall = \frac{TP}{TP + FN}$$

$$Specificity = \frac{TN}{TN + FP}$$

- In other word, sensitivity is the proportion of true positive among all positives
- Specificity is the proportion of false positives among all the negatives

$$Balanced\ accuracy = \frac{Sensitivity + Specificity}{2}$$

- Gives an accuracy values with appropriate weightage to both classes

F1 Score Vs Balanced Accuracy

- F1 score gives more importance to positives in the dataset
- F1 score is ideal with model performance is more associated to identifying positive class (Detecting fraud, disease, fault, etc)
- Balanced accuracy gives importance to both positives and negatives
- To be used when there is equal importance for positives & negatives
- Balanced accuracy can be even used for balanced data while F1 score can mislead on balanced data

Imbalanced data

Predicted Values	Actual Values: Positive	Actual Values: Negative
1	5	10
0	5	980

Balanced accuracy = 0.745
F1 score = 0.4

Balanced data

Predicted Values	Actual Values: Positive	Actual Values: Negative
1	5	10
0	5	0

Balanced accuracy = 0.16
F1 score = 0.4

Scores, Ranking and Thresholds

- Scores:
  - Classifiers output a score $s(\boldsymbol{x})$ for each sample and many times it is $P(y = 1|\boldsymbol{x})$
  - Alternately, it could be distance to decision boundary, log-odds, etc.
- Ranking: Order after sorting by score
  - If sorted by $P(y = 1|\boldsymbol{x})$, we get ranked list of most likely positive to least likely positive samples
- Threshold (τ): Predict positive if $s(x) \geq \tau$
  - If $\tau = 0.7$: TP = 3, FP = 1
  - If $\tau = 0.5$: TP = 4, FP = 3
  - So lowering the thresholds tends to make the classifier more lenient

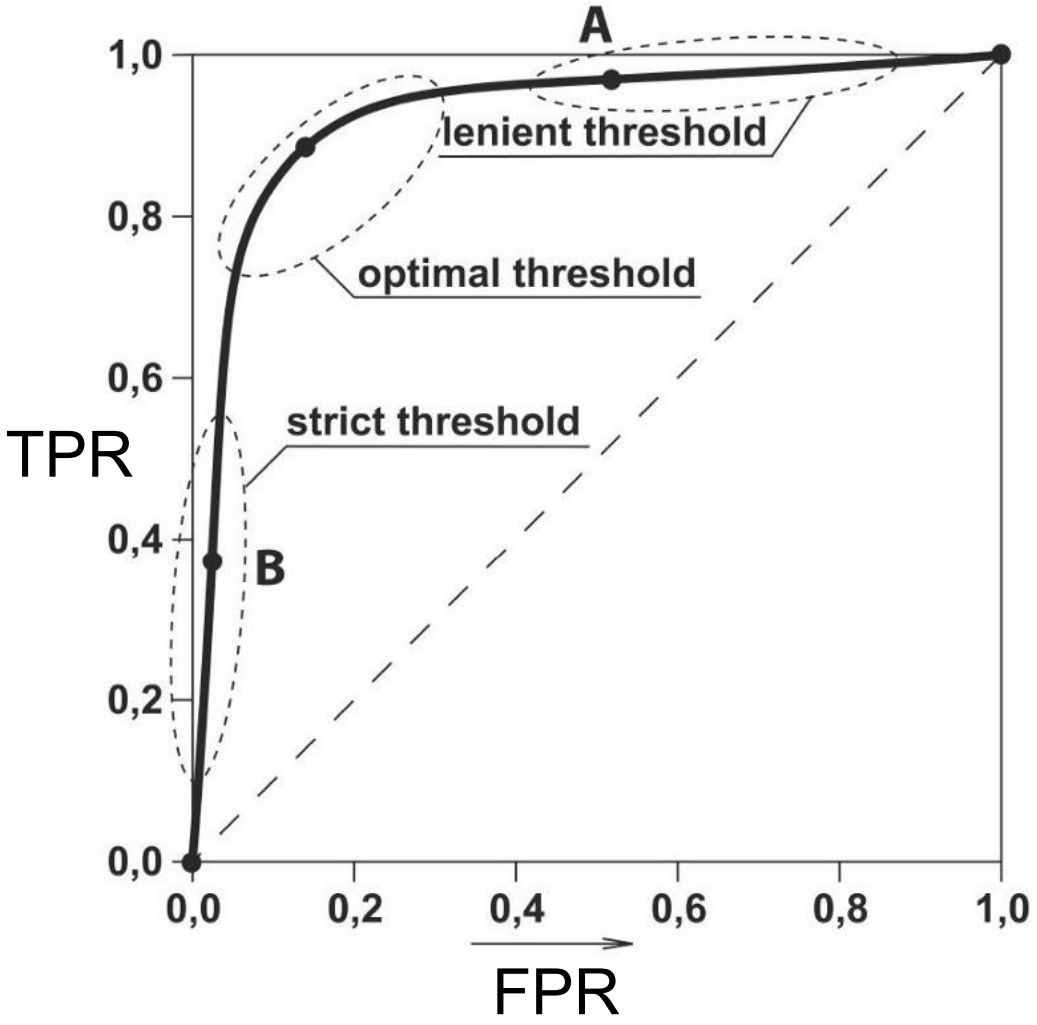
Score	Rank	True Label
0.97	1	P
0.91	2	N
0.88	3	P
0.85	4	P
0.68	5	N
0.65	6	P
0.52	7	N
0.4	8	N
0.32	9	P
0.15	10	N

Performance Metrics: ROC-AUC

- ROC Curve is the plot of True Positive Rate Vs False Positive Rate for varying threshold τ from high to low

$$TPR(\tau) = \frac{TP}{TP + FN} \; ; FPR(\tau) = \frac{FP}{FP + TN}$$

- Good classifiers achieve high TPR at low FPR while random classifier is the diagonal line
- ROC-AUC: Area under the ROC curve summarizes ranking quality across all thresholds
- Choice of τ: Depends on costs of FP and FN
- One heuristic: Maximise $J = TPR - FPR$
- Note: Interpret cautiously when dealing with highly imbalanced data



Naive Baye's Classifier

Motivation

- We have seen two ways to classify:
  - KNN (Non-Parametric): Estimates $P(y = c|\boldsymbol{x})$ from local neighbour frequencies
    $$\hat{y} = \text{argmax}_{\text{c}}\, \hat{P}(y = c|\boldsymbol{x})$$
  - Logistic Regression: Models $P(y = c|\boldsymbol{x})$ directly using sigmoid or softmax functions
    $$P(y = c|\boldsymbol{x}) = \sigma(w_0 + \boldsymbol{w}^T\boldsymbol{x})$$
  - Both the above models are discriminative i.e., predict y given x

- Naive Bayes is generative i.e., it models how data is generated and then uses that model to predict y
  - Learns $P(y)$ and $P(\boldsymbol{x}|y)$ from data
  - Use Baye's rule to compute $P(y|\boldsymbol{x})$

Bayes Classifier and Decision Rule

- Bayes rule

$$P(y = c|\boldsymbol{x}) = \frac{P(\boldsymbol{x}|y = c)P(y = c)}{P(\boldsymbol{x})}$$

- Prior $P(y = c)$: how common class c is
- Likelihood $P(\boldsymbol{x}|y = c)$: how probable it is to see features $\boldsymbol{x}$ if the class were c
- $P(\boldsymbol{x})$: probability of observing the feature vector x under the overall data distribution prior to knowing the class
- For classification, $P(\boldsymbol{x})$ is same for all classes, therefore:

$$\hat{y}(x) = \mathrm{argmax}_{\mathrm{c}}\, P(y = c|\boldsymbol{x}) = \mathrm{argmax}_{\mathrm{c}} P(\boldsymbol{x}|y = c)P(y = c)$$

Naive Bayes Assumption

- Naive Bayes assumes conditional independence of features given the classes
- Let $\boldsymbol{x} = (x_1, \dots, x_D)$

- Naive Assumption: Usually not true but still works in many cases

$$P(\boldsymbol{x}|y = c) = \prod_{j=1}^{D} P(x_j|y = c)$$

- Taking log for numerical stability:

$$\hat{y}(x) = \mathrm{argmax_c}[\log P(\boldsymbol{x}|y = c) + \log P(y = c)]$$

$$\hat{y}(x) = \mathrm{argmax_c}\left[\log P(y = c) + \prod_{j=1}^{D} P\left(x_j|y = c\right)\right]$$

Estimating the Probabilities (Training)

- Given: Training data with N samples $\{x^{(i)}, y^{(i)}\}$
- Estimating the Prior:

$$\hat{P}(y = c) = \frac{N_c}{N}$$

- There are 3 types of naive bayes classifiers based on how the likelihood is computed:
  - Gaussian naive bayes: Works well on continuous features which can take on any numerical value within a range, like temperature, height, weight, or sensor readings
  - Multinomial naive bayes: Works well on discrete features representing counts or frequencies (integer values) like word frequency, page count, etc.
  - Bernoulli naive bayes: Suitable for binary features (0 or 1)

Bernoulli Naive Bayes

- Assumption: Each feature is independent given the class and can be modeled by a Bernoulli distribution
- Likelihood is estimated as follows:

$$P(x_j|y = c) = p_{c,j} \text{ if } x_j = 1$$
$$P(x_j|y = c) = 1 - p_{c,j} \text{ if } x_j = 0$$

where $p_{c,j}$ is the probability that feature x_j is present in class c

- $p_{c,j}$ can be estimated from data as follows:

$$\hat{p}_{c,j} = \frac{N_{c,j} + \alpha}{N_c + 2\alpha}$$

$N_{c,j}$: Number of samples of class c where feature x_j is present

α: Smoothing parameter: handles when a feature is not present in a class

Bernoulli Naive Bayes Example

Document classification based on presence of certain words

# Doc	x1: “urgent”	x2: “sale”	x3: “meeting”	Class
1	1	1	0	Spam (1)
2	0	0	1	Not Spam (0)
3	1	1	0	Spam
4	1	0	1	Not spam
5	0	1	0	Not spam

$$\hat{P}(y = c) = \frac{N_c}{N}$$

$$\hat{p}_{c,j} = \frac{N_{c,j} + \alpha}{N_c + 2\alpha}$$

$N_{c,j}$: Number of samples of class c where feature x_j is present

$\hat{P}(y=0) = 3/5$ $\hat{P}(y=1) = 2/5$

$\hat{P}_{1,1} = \frac{2+1}{2+2} = 3/4$ $\hat{P}_{0,3} = 3/5$

Multinomial Naive Bayes

- Assumption: Assumes features follow a multinomial distribution i.e., probabilities of each feature value sum up to 1

- Likelihood is estimated as follows:
  - Estimate class-wise feature probabilities from counts
    - $N_{c,j}$: Total count of feature x_j across samples in class c
    - N_c: Total count across all features for class c

$$\hat{\theta}_{c,j} = \frac{N_{c,j} + \alpha}{N_{c,.} + \alpha . D}$$

$$P(x_j | y = c) \propto \theta_{c,j}^{x_j}$$

Multinomial Naive Bayes Example

Suppose we have two classes and three count features

Sample	x1	x2	x3	Class
1	2	1	0	A
2	1	2	1	A
3	0	1	3	B
4	1	0	2	B
5	0	2	1	B

$$\hat{P}(y = c) = \frac{N_c}{N}$$

$$\hat{\theta}_{c,j} = \frac{N_{c,j} + \alpha}{N_{c,.} + \alpha . D}$$

$\hat{P}(y=A) = 2/5$ $\hat{P}(y=B) = 3/5$

$\hat{\theta}_{A,1} = \frac{3+1}{7+3} = \frac{4}{10}$ $\hat{\theta}_{A,2} = \frac{4}{10}$ $\hat{\theta}_{A,3} = \frac{2}{10}$

Gaussian Naive Bayes

- Assumption: For each class c and each feature j, $x_j|y = c$ is Gaussian and features are conditionally independent given the class
- Likelihood is estimated as follows:

$$P(x_j \mid y = c) = \frac{1}{\sqrt{2\pi\sigma_{c,j}^2}} \exp\left(-\frac{(x_j - \mu_{c,j})^2}{2\sigma_{c,j}^2}\right)$$

$$\hat{\mu}_{c,j} = \frac{1}{N_c} \sum_{i:y_i=c} x_{i,j}$$

$$\hat{\sigma}_{c,j} = \frac{1}{N_c} \sum_{i:y_i=c} (x_{i,j} - \hat{\mu}_{c,j})^2$$

Naive Bayes: Prediction Steps

- At the end of training we have:
  - Prior for each class $\hat{P}(y = c)$
  - Likelihood parameters for each feature j
- Prediction for a new sample x_{new}:
  - Step 1: Compute the class scores

$$s(y=c) = \log \hat{p}(y=c) + \sum_{j=1}^{D} \log \hat{p}(x_j \mid y=c)$$

  - Step 2: Choose the class with maximum score

$$\hat{y} = \underset{c}{\operatorname{argmax}}\ s(y=c)$$

  - Step 3: (Optional) Compute normalised posterior probabilities

$$\hat{p}(y=c \mid x) = \frac{\exp(s(y=c))}{\sum_{c} \exp(s(y=c))}$$

Score Computation

- Bernoulli Naive Bayes:

$$\log p(x_j \mid y=c) = \underbrace{x_j}_{x_{j,new}} \log \underbrace{p_{c,j}} + (1-x_j)\log(1-p_{c,j})$$

- Multinomial Naive Bayes:

$$\log p(x_j \mid y=c) = x_j \log \theta_{c,j}$$

- Gaussian Naive Bayes:

$$\log p(x_j \mid y=c) = -\frac{1}{2}\left[\log(2\pi\sigma_{c,j}^2) + \frac{(x_j-\mu_{c,j})^2}{\sigma_{c,j}^2}\right]$$

New sample

Handling Data Imbalance in Classification

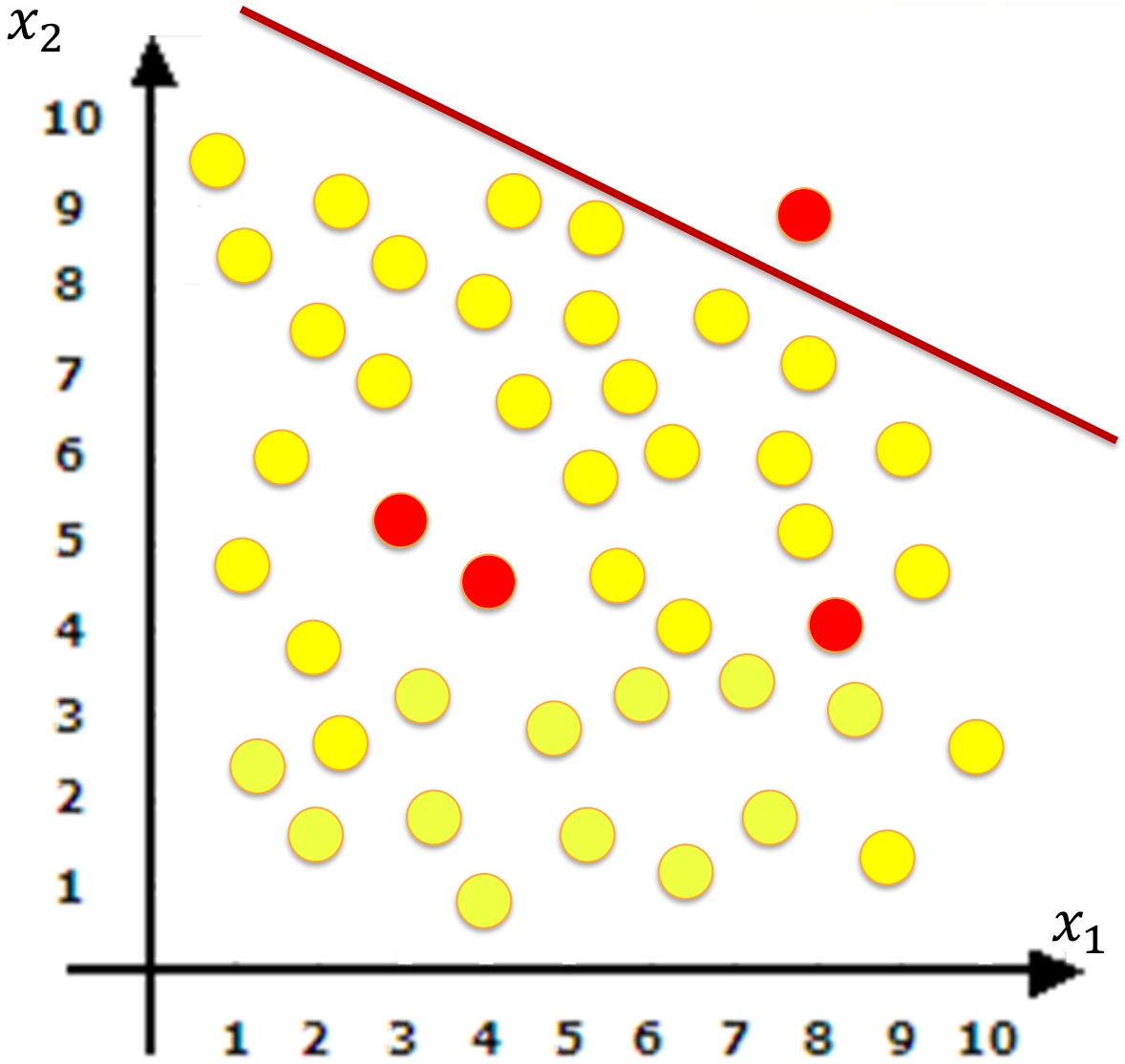
Handling Data Imbalance in Classification

- Data imbalance refers to the scenario where the number of samples belonging to each class are skewed (majority class and minority class)
- In binary classification problem, data is imbalanced if

  N_0 (No. of training samples of class 0) >> N_1 (No. of training samples of class 1)
- Examples of problems where data is generally imbalanced:
  - Fraud detection in financial transactions (Genuine transactions >>> Fraud transactions)
  - Predicting whether a tumour is malignant or not based on diagnostic reports
  - Identifying faulty condition in a process or a system of an industry
  - Spam detection in emails

Handling Data Imbalance in Classification

- Data imbalance significantly affects the performance of classification model learnt
- Accuracy would be high but model performance on the minority class would be very poor
- Techniques such as undersampling, oversampling, synthetic sampling, weighing samples, etc. need to be used to handle data imbalance
- Performance measures such as precision recall, F1 score or balanced accuracy need to be used instead of accuracy for model validation



Issue of Data Imbalance

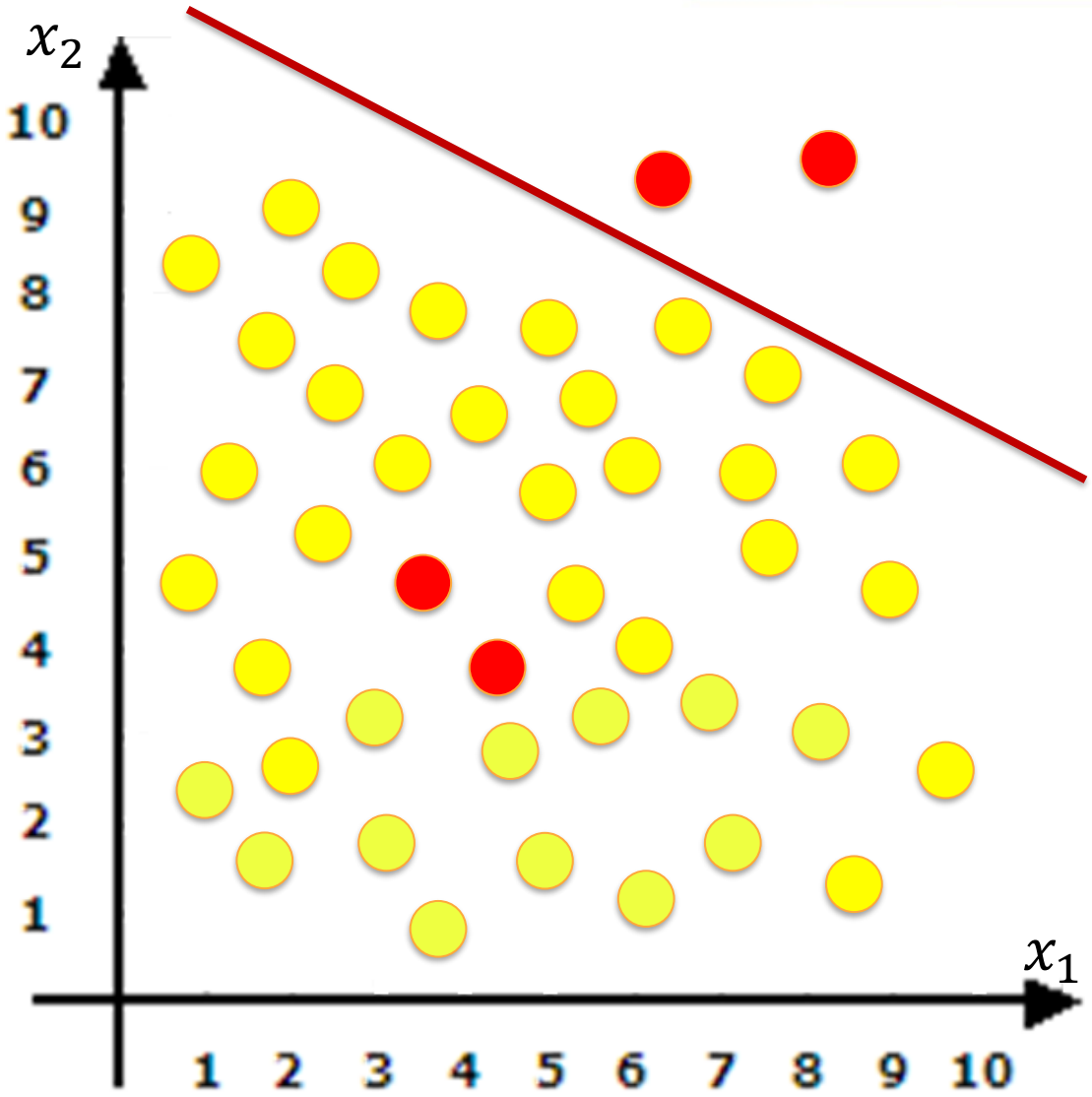
- Impact on Loss: Impact on loss is minimal when minority samples are misclassified
- In other words, majority samples dominate the loss computation and model prioritises to minimise errors on majority class resulting in bias
- In such a case, give more weightage to minority class samples in loss function

$$log\ loss = -\frac{1}{N}\sum_{i} w_i(y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i))$$

$$w_1 = \frac{N_0 + N_1}{N_1 * 2}\ ; w_0 = \frac{N_0 + N_1}{N_0 * 2}$$

$$N_0 = 900\ ; N_1 = 100$$

$$w_1 = \frac{1000}{200} = 5\ ; w_0 = \frac{1000}{1800} = 0.55$$



Issue of Data Imbalance

- Other ways to Handle Data Imbalance:
  - Changing the probability threshold of predictions
    - Lower threshold can be set for minority class
  - Under-sampling the majority class
  - Over-sampling or synthesis sampling the minority class (SMOTE, ADASYN, etc.)

Summary

- ✓ Classifiers are build to make predictions on class labels and they divide the feature space into different regions which are assigned different classes
- ✓ Classifiers can be categories into different types based on nature of decision boundary and number of classes
- ✓ kNN predicts the class of a new sample based on the majority class of nearest k neighbours
- ✓ Logistic regression applies a sigmoid or softmax function on linear combination of inputs to predict class probabilities
- ✓ Naïve Bayes models the distribution of the data and uses Bayes rule to predict probability of output
- ✓ Balanced accuracy to be preferred as evaluation metric in general
- ✓ If training data pertaining to different samples is skewed, then the skew needs to be handled before building a classification model to ensure good performance

operation == "MIRROR_X":
mirror_mod.use_x = True
mirror_mod.use_y = False
mirror_mod.use_z = False
_operation == "MIRROR_Y":
mirror_mod.use_x = False
mirror_mod.use_y = True
mirror_mod.use_z = False
_operation == "MIRROR_Z":
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True
selection at the end -add
_ob.select= 1
ier_ob.select=1
ntext.scene.objects.active
("Selected" + str(modifier
irror_ob.select = 0
bpy.context.selected_obj
ata.objects[one.name].sel
int("please select exactly

THANK YOU